Software Requirements Specification

Hospital Appointment Management System

Version: 1.0

Date: March 1, 2026

Prepared By: Senior Software Architect

Standard: IEEE Std 830-1998

Table of Contents

1. Introduction
- **1.1** Purpose
- **1.2** Scope
- **1.3** Definitions, Acronyms, and Abbreviations
- **1.4** References
- **1.5** Document Overview

2. Overall Description
- **2.1** Product Perspective
- **2.2** Product Functions
- **2.3** User Classes and Characteristics
- **2.4** Operating Environment
- **2.5** Design and Implementation Constraints
- **2.6** Assumptions and Dependencies

3. Functional Requirements
- **3.1** FR 1 – Patient Registration
- **3.2** FR 2 – User Authentication and Login
- **3.3** FR 3 – Doctor Management
- **3.4** FR 4 – Appointment Booking
- **3.5** FR 5 – Appointment Cancellation
- **3.6** FR 6 – Doctor Availability Management
- **3.7** FR 7 – Appointment History
- **3.8** FR 8 – Admin System Management

4. Non-Functional Requirements
- **4.1** Performance
- **4.2** Security
- **4.3** Usability
- **4.4** Reliability
- **4.5** Portability

5. Use Case Diagram

6. Workflow Activity Diagram

7. Data Flow Diagram (Level 1)

1. Introduction

1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) is to provide a comprehensive and unambiguous definition of the functional and non-functional requirements for the **Hospital Appointment Management System**. This web-based system enables patients to register, authenticate, and schedule appointments with physicians, while supporting role-specific access controls for Patients, Doctors, and Administrators.

This document is intended to serve all project stakeholders — including developers, quality assurance engineers, project managers, and end-users — as a contractual baseline for development and acceptance testing. By adhering to the IEEE Std 830-1998 standard, the SRS facilitates verification and validation processes, minimises ambiguities, and prevents scope creep.

1.2 Scope

The Hospital Appointment Management System is a multi-role, web-based platform designed to streamline patient-doctor appointment workflows. The system provides three distinct interfaces: a **Patient portal** for self-service scheduling, a **Doctor portal** for appointment management and schedule control, and an **Admin console** for system-wide oversight and reporting.

The scope explicitly excludes telemedicine features, billing integration, prescription management, dedicated native mobile applications, and integration with external healthcare APIs.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
IEEE	Institute of Electrical and Electronics Engineers
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
UI	User Interface
DFD	Data Flow Diagram
FR	Functional Requirement
NFR	Non-Functional Requirement
HTTPS	Hypertext Transfer Protocol Secure
MVC	Model-View-Controller
Admin	System Administrator with full system oversight privileges
Patient	Registered end-user who books appointments
Doctor	Medical professional with schedule management access

1.4 References

Ref.	Source	Description
[1]	IEEE Std 830-1998	IEEE Recommended Practice for Software Requirements Specifications
[2]	Bootstrap 5 Documentation	https://getbootstrap.com/docs/5.0/
[3]	Node.js Documentation	https://nodejs.org/en/docs/
[4]	MySQL 8.0 Reference Manual	https://dev.mysql.com/doc/refman/8.0/en/
[5]	OWASP Security Guidelines	https://owasp.org/www-project-top-ten/

1.5 Document Overview

This SRS is structured into seven major sections. Section 1 introduces the document's purpose, scope, terminology, and references. Section 2 provides an overall product description. Section 3 details all functional requirements. Section 4 covers non-functional requirements. Sections 5, 6, and 7 present the Use Case Diagram, Workflow Activity Diagram, and Level-1 Data Flow Diagram respectively, each followed by an explanation based strictly on the diagrams provided.

2. Overall Description

2.1 Product Perspective

The Hospital Appointment Management System is a standalone, web-based application operating within a three-tier client-server architecture. It functions as a dedicated scheduling and appointment coordination module. The system interfaces with a centralised MySQL relational database for persistent storage of patient, doctor, and appointment records.

The system follows the Model-View-Controller (MVC) pattern. The View layer consists of server-rendered HTML pages styled with Bootstrap 5. The Controller layer is implemented using Node.js and Express.js. The Model layer manages database interactions through MySQL. Authentication is secured via JSON Web Tokens (JWT).

Layer / Component	Technology
Frontend	HTML5, CSS3, JavaScript (ES6+), Bootstrap 5
Backend	Node.js v18+, Express.js v4+
Database	MySQL 8.0
Authentication	JWT / Express Session + bcrypt
Deployment	AWS EC2 / Heroku / DigitalOcean
Version Control	Git / GitHub

2.2 Product Functions

The system provides the following core functional capabilities, grouped by user role:

Role	Functional Capabilities
Patient	Registration, Login/Logout, Search Doctor by Specialization, View Doctor List, Book Appointment, Cancel Appointment, View Appointment History, Update Profile
Doctor	Login/Logout, View Daily Appointments, Accept/Reject Appointment Requests, Update Availability Schedule, View Patient Details
Admin	Login/Logout, Manage Doctors, Manage Patients, View All Appointments, Generate System Reports, Manage System Settings

2.3 User Classes and Characteristics

Role	Technical Proficiency	Permissions Granted	Restrictions
Patient (User)	Low–Moderate	Register account; login/logout; search and view doctors; book and cancel appointments; view appointment history; update profile	No access to other patients' data; no admin or doctor functions
Doctor	Moderate	Login/logout; view daily appointments; accept or reject appointment requests; update availability schedule; view patient details	Cannot access admin functions; limited to own schedule and assigned patients

Role	Technical Proficiency	Permissions Granted	Restrictions
Admin	High	Manage doctors; manage patients; view all appointments; generate system reports; manage system settings	Full system access; cannot impersonate patients or doctors

2.4 Operating Environment

Component	Specification
Client Side	Modern web browsers (Chrome 100+, Firefox 100+, Edge 100+, Safari 15+) on desktop and mobile devices
Server Side	Node.js v18+ on Linux-based cloud server (Ubuntu 22.04 LTS recommended)
Database	MySQL 8.0, minimum 4 GB RAM, 50 GB storage
Network	HTTPS (TLS 1.2+) for all client-server communications
Deployment	Cloud infrastructure (AWS EC2, Heroku, or DigitalOcean)

2.5 Design and Implementation Constraints

Constraint Type	Description
Technology Stack	Mandatory use of Node.js, Express.js, MySQL, and Bootstrap 5
Authentication	JWT or Express-Session with bcrypt; no plaintext password storage
RBAC Enforcement	Role-based access enforced server-side for all API endpoints
Data Isolation	Patient data scoped by user ID; cross-patient data access prohibited
Responsive Design	All UI must render correctly on widths from 320px to 2560px
Web-Only Deployment	No native mobile or desktop application; web-only access
Session Security	Sessions expire after 30 minutes of inactivity; HTTPS mandatory in production

2.6 Assumptions and Dependencies

Assumptions:

ID	Assumption
A1	All users have stable internet access and modern browsers with JavaScript enabled
A2	The MySQL server is available and properly configured
A3	End-users possess basic web literacy for navigation
A4	Doctors actively manage their availability schedules within the system
A5	The system is deployed in a secure HTTPS-enabled environment in production

Dependencies:

ID	Dependency
D1	NPM packages: express, mysql2, bcryptjs, jsonwebtoken, express-session, ejs
D2	Browser compatibility with Chrome 100+, Firefox 100+, Edge 100+, Safari 15+
D3	Cloud hosting platform availability and uptime SLA

3. Functional Requirements

Each functional requirement is documented using a standardised template comprising: **Description**, **Inputs**, **System Behaviour**, and **Outputs**.

3.1 Patient Registration

Description: The system shall allow new patients to create a personal account by providing identification and contact details. Upon successful registration the patient can log into the system.

Inputs:

Field	Type	Validation	Required
Full Name	String (max 100)	Non-empty; alphabetic	Yes
Email	String (email)	Valid format; must be unique	Yes
Password	String (min 8)	Min 8 chars; complexity rules	Yes
Phone Number	String (10–15 d.)	Numeric; valid phone format	Yes
Date of Birth	Date YYYY-MM-DD	Must be in the past	Yes
Gender	Enum M/F/Other	One of the defined values	Yes
Address	String (max 255)	Optional descriptive text	Optional

System Behaviour:

1. System validates all required fields for completeness and format.
2. System queries the Patient Database to verify email uniqueness.
3. System hashes the provided password using bcrypt.
4. System inserts a new patient record with a generated unique Patient ID.
5. System redirects the patient to the login page upon successful registration.

Outputs:

- Success: 'Account created successfully. Please log in.'
- Error (duplicate email): 'An account with this email already exists.'
- Error (validation): Field-level error messages for each failed rule.
- New patient record persisted in the Patient Database.

3.2 User Authentication and Login

Description: The system shall authenticate all users (Patients, Doctors, Admins) and redirect them to their respective role-specific dashboards. Failed attempts are tracked.

Inputs:

Field	Type	Validation	Required
Email / Username	String	Must match a registered account	Yes

Field	Type	Validation	Required
Password	String	Must match the stored bcrypt hash	Yes
Role	Auto-detected (DB)	System determines role from record	N/A

System Behaviour:

1. System retrieves the user record matching the provided email.
2. System validates the submitted password against the stored bcrypt hash.
3. If credentials are invalid, an error message is displayed.
4. If five consecutive failures occur, the account is temporarily locked for 15 minutes.
5. On success, a JWT token / session is created with role and user ID claims.
6. Patient → Patient Dashboard; Doctor → Doctor Dashboard; Admin → Admin Console.
7. Session expiry is set to 30 minutes of inactivity.

Outputs:

- Success: Role-specific dashboard rendered; session/JWT token issued.
- Error: 'Invalid email or password. Please try again.'
- Account lockout: 'Too many failed attempts. Account locked for 15 minutes.'
- System log entry created for each authentication attempt.

3.3 Doctor Management

Description: The Admin shall be able to add new doctors, edit existing doctor profiles, and activate or deactivate doctor accounts. Changes are immediately reflected in patient-facing listings.

Inputs:

Field	Type	Validation	Required
Doctor Full Name	String (max 100)	Non-empty; alphabetic	Yes
Email Address	String (email)	Valid email; unique in Doctor DB	Yes
Specialization	String / Enum	Must match a defined specialty	Yes
Qualifications	String (max 500)	Non-empty descriptive text	Yes
Experience (Years)	Integer >= 0	Non-negative integer	Yes
Consultation Fee	Decimal (>= 0)	Non-negative numeric value	Yes
Status	Enum Active/Inactive	Defaults to Active on creation	Yes

System Behaviour:

1. Admin navigates to the Doctor Management section of the Admin Console.
2. For Create: system validates fields, checks email uniqueness, assigns Doctor ID, inserts record.
3. For Edit: system retrieves existing record by Doctor ID, applies validated changes.

4. For Deactivate: system sets status to 'Inactive'; doctor removed from patient search.
5. All changes are logged in the Audit Log with timestamp and Admin ID.

Outputs:

- Success (Create): 'Doctor account created successfully.'
- Success (Edit): 'Doctor profile updated successfully.'
- Success (Deactivate): 'Doctor account deactivated.'
- Error (duplicate email): 'A doctor with this email already exists.'
- Audit log entry created for every admin action.

3.4 Appointment Booking

Description: The system shall allow authenticated patients to book an appointment with a selected doctor on an available date and time slot, preventing double-booking.

Inputs:

Field	Type	Validation	Required
Doctor ID	Integer (FK)	Must reference an Active doctor	Yes
Appointment Date	Date YYYY-MM-DD	Future date within doctor's availability	Yes
Time Slot	Time Enum	Must be an available slot	Yes
Reason for Visit	String (max 500)	Non-empty descriptive text	Yes
Patient ID	Integer (auto session)	Authenticated patient's ID	Auto

System Behaviour:

1. Patient selects a doctor from search results or the doctor listing.
2. System retrieves and displays the doctor's available time slots for the chosen date.
3. Patient selects date, time slot, and enters the reason for visit.
4. System verifies the slot has not been concurrently booked (atomic DB transaction).
5. System creates an appointment record with status 'Pending' in the Appointment Database.
6. Doctor receives an in-system notification of the new appointment request.

Outputs:

- Success: 'Appointment request submitted. Awaiting doctor confirmation.' Status = Pending.
- Error (slot taken): 'The selected time slot is no longer available.'
- Error (past date): 'Appointment date must be in the future.'
- Appointment record persisted; doctor notification queued.

3.5 Appointment Cancellation

Description: The system shall allow authenticated patients to cancel a pending or confirmed appointment subject to a minimum 2-hour cancellation window. The freed slot becomes available.

Inputs:

Field	Type	Validation	Required
Appointment ID	Integer (FK)	Must belong to the logged-in patient	Yes
Cancellation Reason	String (max 255)	Optional descriptive text	Optional
Patient ID	Integer (auto session)	Must match appointment's Patient ID	Auto

System Behaviour:

1. Patient navigates to 'My Appointments' and selects the appointment to cancel.
2. System verifies ownership (appointment belongs to logged-in patient).
3. System checks that status is 'Pending' or 'Confirmed' (not 'Completed'/'Cancelled').
4. System validates that current time is at least 2 hours before appointment time.
5. System updates appointment status to 'Cancelled'.
6. Freed time slot is marked available in the doctor's schedule.
7. Cancellation notification sent to the doctor.

Outputs:

- Success: 'Appointment cancelled successfully.' Status updated to 'Cancelled'.
- Error (too late): 'Cannot cancel within 2 hours of the scheduled time.'
- Error (wrong status): 'This appointment cannot be cancelled.'
- Doctor notified; time slot reopened for rebooking.

3.6 Doctor Availability Management

Description: The system shall allow authenticated doctors to define their weekly availability schedule, including working hours and slot durations. Doctors may block specific dates for leave.

Inputs:

Field	Type	Validation	Required
Doctor ID	Integer (auto session)	Authenticated doctor's ID	Auto
Day of Week	Enum Mon–Sun	Valid day selection	Yes
Start Time	Time HH:MM	Must be earlier than End Time	Yes
End Time	Time HH:MM	Must be later than Start Time	Yes
Slot Duration (min)	Integer 15/20/30/60	One of the allowed durations	Yes
Leave Date	Date YYYY-MM-DD	Future date to block availability	Optional

System Behaviour:

1. Doctor navigates to 'My Schedule' section of the Doctor Dashboard.
2. Doctor configures available days, start/end times, and slot duration.
3. System validates time consistency (Start Time < End Time).

4. System persists the schedule in the Doctor Schedule Database.
5. For leave blocking: doctor selects a date; system marks all slots on that date as unavailable.
6. Updated availability immediately reflected in the patient-facing booking interface.

Outputs:

- Success: 'Availability schedule updated successfully.'
- Success (leave block): 'Date marked as unavailable.'
- Error (time conflict): 'End Time must be later than Start Time.'
- Patient booking interface refreshed with new availability.

3.7 Appointment History

Description: The system shall allow patients to view their complete appointment history. Doctors may view their own appointment history. The view supports filtering and sorting.

Inputs:

Field	Type	Validation	Required
User ID	Integer (auto session)	Authenticated user's ID	Auto
Filter: Status	Enum All/Pending/etc.	Valid status value	Optional
Filter: Date	Date range From–To	From <= To	Optional
Sort By	Enum Date Asc/Desc	Valid sort order	Optional

System Behaviour:

1. User navigates to 'Appointment History' from their dashboard.
2. System queries the Appointment Database scoped by the user's ID and role.
3. Applied filters refine the query (WHERE clauses on status, date range).
4. Results sorted by specified order (default: date descending).
5. System renders a paginated table of appointment records (20 per page).

Outputs:

- Paginated appointment history table rendered on screen.
- Each row shows: Appointment ID, Date/Time, Doctor/Patient Name, Status, Reason.
- Total count displayed: 'Showing X of Y appointments.'
- Empty state: 'No appointments found for the selected criteria.'

3.8 Admin System Management

Description: The Admin shall have access to a comprehensive management console providing system-wide visibility and control — including user management, appointment oversight, report generation, and system configuration.

Inputs:

Field	Type	Validation	Required
Report Type	Enum Daily/Weekly/Monthly	Valid report type selection	Yes

Field	Type	Validation	Required
Date Range	Date pair From–To	From <= To; both valid dates	Conditional
Search Term	String	Partial match on name/email	Optional
Setting Key	String	Defined setting identifier	Conditional
Setting Value	Varies	Validated per setting spec	Conditional

System Behaviour:

1. Admin authenticates and is directed to the Admin Console dashboard.
2. Dashboard displays KPIs: total patients, total doctors, appointments today, pending requests.
3. For Doctor/Patient Management: admin can search, view, edit, activate, or deactivate records.
4. For All Appointments: admin views a global appointment list with full filtering.
5. For Report Generation: admin selects type and date range; system generates report as HTML/CSV.
6. For System Settings: admin modifies configuration parameters; changes persisted immediately.
7. All admin actions recorded in the Audit Log with timestamp, Admin ID, and action description.

Outputs:

- Admin Dashboard with live statistics rendered.
- Filtered and paginated tables for user and appointment management.
- Generated reports available as on-screen view and downloadable CSV.
- Success messages for all management actions.
- Comprehensive audit log entries for every admin operation.

4. Non-Functional Requirements

4.1 Performance

Metric	Context	Target	Basis
Page Load Time	Dashboard and listing pages	≤ 2 seconds	95th percentile
API Response Time	Standard CRUD operations	≤ 500 ms	99th percentile
Concurrent Users	Simultaneous active sessions	≥ 500 users	Without degradation
Search Response	Doctor search by specialization	≤ 300 ms	With indexing
Report Generation	Monthly appointment reports	≤ 5 seconds	Up to 10,000 records

4.2 Security

ID	Control	Requirement
SC-01	Password Hashing	bcrypt with ≥ 12 salt rounds; no plaintext passwords stored
SC-02	Authentication Tokens	JWT signed with RS256; HttpOnly and Secure flags on session cookies
SC-03	Server-Side RBAC	All API routes validate role claims at middleware level
SC-04	Input Validation	All inputs sanitised server-side; parameterised queries throughout
SC-05	SQL Injection	Exclusively parameterised SQL or ORM; no raw query concatenation
SC-06	XSS Prevention	All HTML output escaped; Content Security Policy headers enabled
SC-07	CSRF Protection	CSRF tokens required on all state-changing form submissions
SC-08	Account Lockout	Temporary lock after 5 consecutive failed login attempts (15 min)
SC-09	Session Expiry	Sessions expire after 30 minutes of inactivity
SC-10	HTTPS Enforcement	All production traffic served over TLS 1.2+

4.3 Usability

Criterion	Requirement
Responsive Design	All pages must function on screen widths from 320px to 2560px
Learnability	A new patient should register and book an appointment within 5 minutes without external help
Feedback	Every user action must produce clear success or error feedback within 2 seconds
Navigation	All primary features accessible within 3 clicks from the role-specific dashboard
Accessibility	Colour contrast ratio ≥ 4.5:1 (WCAG AA); keyboard navigation supported
Loading Indicators	Spinner displayed for operations expected to take > 500 ms

4.4 Reliability

Attribute	Requirement
System Uptime	99.5% monthly uptime (≤ 3.65 hours downtime per month)
Data Integrity	All transactions are ACID-compliant; no partial writes permitted
Data Backup	Automated daily MySQL backups retained for 30 days
Failover	PM2 process manager with automatic restart on application crash
Graceful Degradation	If a non-critical service fails, core booking functionality remains available

4.5 Portability

Attribute	Requirement
Browser Compatibility	Chrome 100+, Firefox 100+, Edge 100+, Safari 15+; IE not supported
Device Compatibility	Desktop (Windows, macOS, Linux), tablet, and mobile web browsers
Cloud Portability	Deployable on AWS, GCP, Azure, Heroku, or DigitalOcean with minimal config change
Container Support	Application structure compatible with Docker containerisation

5. Use Case Diagram

The diagram below presents the Use Case Diagram for the Hospital Appointment Management System. It identifies three actors — **Patient**, **Doctor**, and **Admin** — and the complete set of use cases each actor interacts with, grouped by functional area.

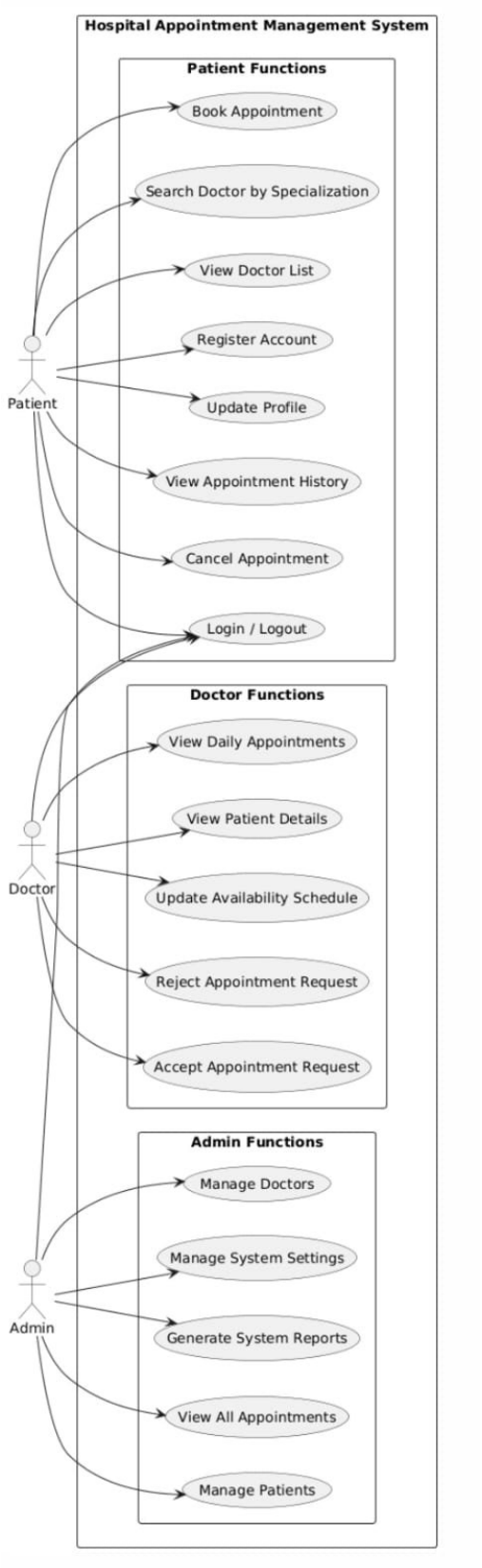

Figure 5.1 – Hospital Appointment Management System: Use Case Diagram

System Boundary:

All use cases are enclosed within the **Hospital Appointment Management System** boundary. Three actors — Patient, Doctor, and Admin — interact with the system from outside the boundary.

Patient Actor – Use Cases (Patient Functions):

Use Case	Description as Shown in Diagram
Book Appointment	Patient initiates a new appointment booking with a doctor
Search Doctor by Specialization	Patient searches the doctor list filtered by medical specialty
View Doctor List	Patient browses all available doctors in the system
Register Account	Patient creates a new account in the system
Update Profile	Patient modifies personal profile information
View Appointment History	Patient views all past and upcoming appointments
Cancel Appointment	Patient cancels a previously booked appointment
Login / Logout	Patient authenticates into the system and terminates the session

Doctor Actor – Use Cases (Doctor Functions):

Use Case	Description as Shown in Diagram
View Daily Appointments	Doctor views all appointments scheduled for the current day
View Patient Details	Doctor views the profile and details of a scheduled patient
Update Availability Schedule	Doctor defines or modifies working hours and available slots
Reject Appointment Request	Doctor declines a pending appointment request from a patient
Accept Appointment Request	Doctor confirms and accepts a pending appointment request

Admin Actor – Use Cases (Admin Functions):

Use Case	Description as Shown in Diagram
Manage Doctors	Admin adds, edits, activates, or deactivates doctor accounts
Manage System Settings	Admin configures system-wide parameters and settings
Generate System Reports	Admin generates operational and statistical reports
View All Appointments	Admin accesses a global view of all system appointments
Manage Patients	Admin views and manages patient records

6. Workflow Activity Diagram

The diagram below presents the Workflow Activity Diagram for the Hospital Appointment Management System. It illustrates the complete operational flow from system start through role-based activity execution to system end.

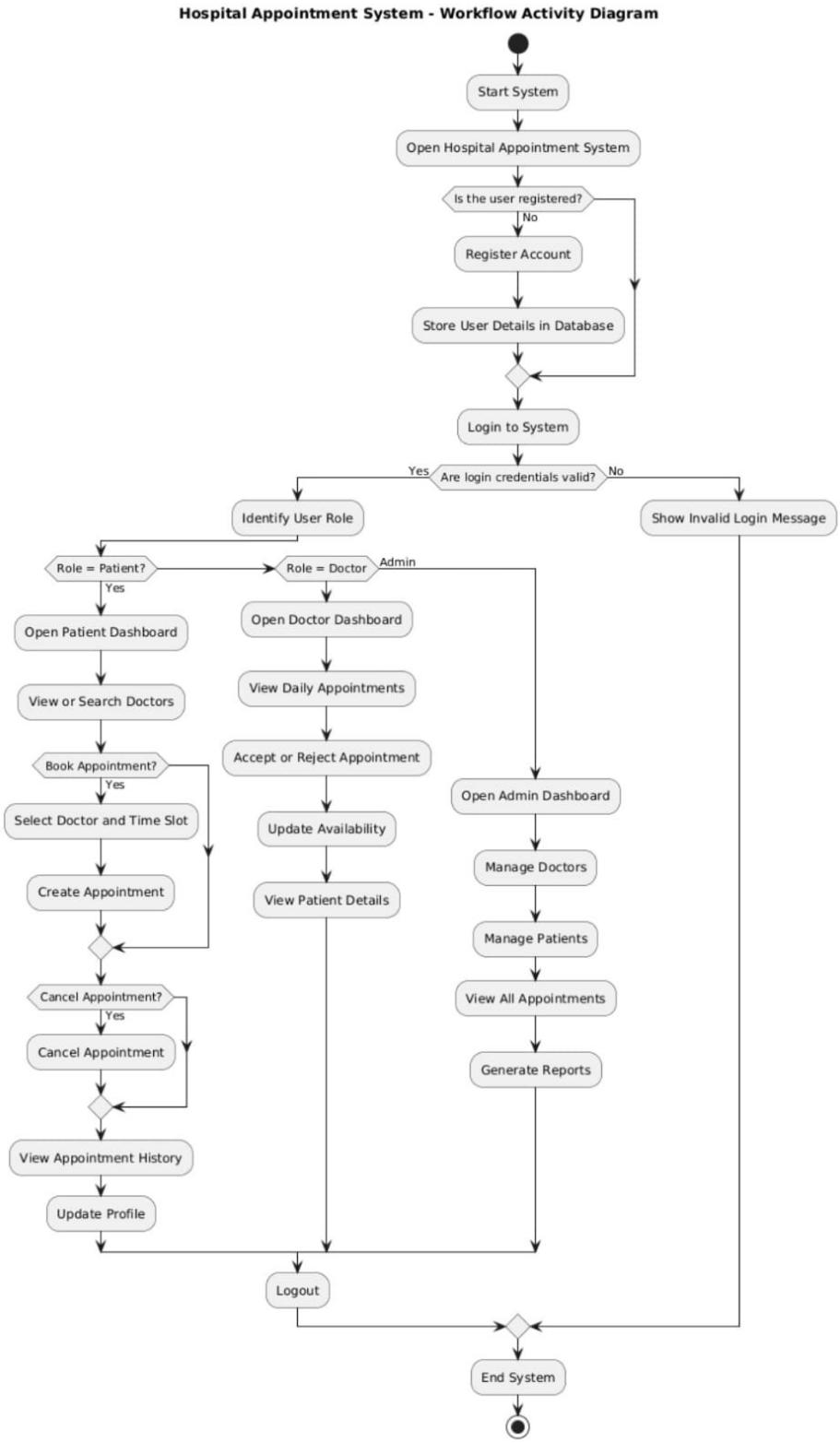

Figure 6.1 – Hospital Appointment Management System: Workflow Activity Diagram

Workflow Explanation (as depicted in Figure 6.1):

Step 1 – System Start

The workflow begins with the **Start System** initial node, followed immediately by **Open Hospital Appointment System**, which launches the application.

Step 2 – Registration Check

The system presents the decision: **'Is the user registered?'** If **No**, the flow proceeds to **Register Account**, which stores the new user's details in the database via **Store User Details in Database**. The flow then merges back to the login step. If **Yes**, the user proceeds directly to login.

Step 3 – Login and Credential Validation

The user performs **Login to System**. The system evaluates: **'Are login credentials valid?'** If **No**, the system shows **Show Invalid Login Message** and the workflow branches away. If **Yes**, the flow proceeds to **Identify User Role**.

Step 4 – Role Identification and Dashboard Routing

The system identifies the authenticated user's role. Three parallel branches are shown: • **Role = Patient:** Opens Patient Dashboard • **Role = Doctor:** Opens Doctor Dashboard • **Admin:** Opens Admin Dashboard

Step 5 – Patient Workflow

From the **Patient Dashboard**, the patient follows this sequence: **View or Search Doctors** → decision **'Book Appointment?'** → If Yes: **Select Doctor and Time Slot** → **Create Appointment**. Next decision: **'Cancel Appointment?'** → If Yes: **Cancel Appointment**. The flow then proceeds to **View Appointment History** → **Update Profile** → **Logout**.

Step 6 – Doctor Workflow

From the **Doctor Dashboard**, the doctor follows: **View Daily Appointments** → **Accept or Reject Appointment** → **Update Availability** → **View Patient Details** → **Logout**.

Step 7 – Admin Workflow

From the **Admin Dashboard**, the admin follows: **Manage Doctors** → **Manage Patients** → **View All Appointments** → **Generate Reports** → **Logout**.

Step 8 – End System

All three role branches converge at a merge node leading to the **End System** terminal node, completing the workflow.

7. Data Flow Diagram (Level 1)

The diagram below presents the Level-1 Data Flow Diagram (DFD) for the Hospital Appointment Management System. It shows four processes, three data stores, and three external entities, along with the data flows connecting them.

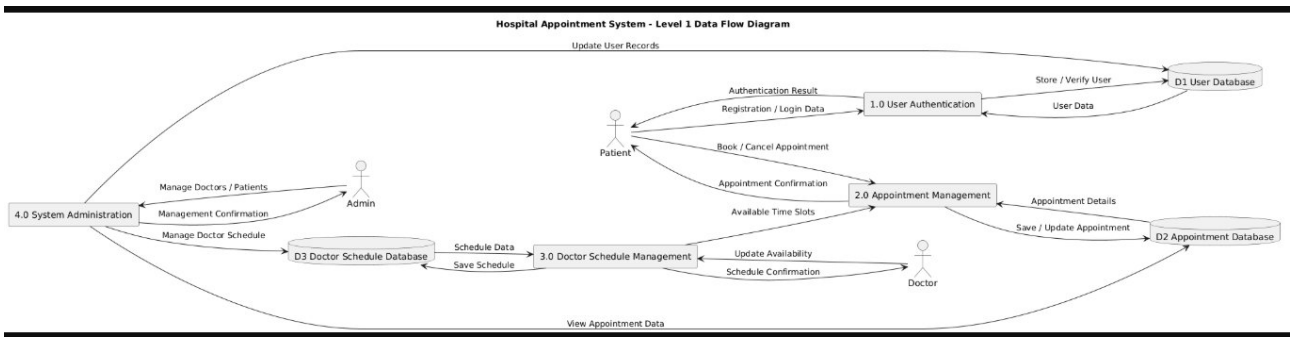

Figure 7.1 – Hospital Appointment System: Level 1 Data Flow Diagram

DFD Explanation (as depicted in Figure 7.1):

External Entities:

External Entity	Role and Data Flows as Shown in Diagram
Patient	Primary external entity. Sends Registration/Login Data to Process 1.0 and Book/Cancel Appointment requests to Process 2.0. Receives Authentication Result and Appointment Confirmation / Available Time Slots from the system. Also receives Update User Records from Process 4.0.
Doctor	External entity interacting with Process 3.0. Sends Update Availability to the Doctor Schedule Management process. Receives Schedule Confirmation from the same process.
Admin	External entity interacting with Process 4.0 (System Administration). Sends Manage Doctors/Patients and Manage Doctor Schedule commands. Receives Management Confirmation. Also sends the View Appointment Data flow at the bottom of the diagram.

Processes:

Process	Data Flows as Shown in Diagram
1.0 User Authentication	Receives Registration/Login Data from Patient. Interacts with D1 User Database (Store/Verify User; receives User Data back). Returns Authentication Result to the Patient.
2.0 Appointment Management	Receives Book/Cancel Appointment from Patient. Returns Appointment Confirmation and Available Time Slots to Patient. Interacts with D2 Appointment Database (Save/Update Appointment; receives Appointment Details).
3.0 Doctor Schedule Management	Receives Update Availability from Doctor and returns Schedule Confirmation. Interacts with D3 Doctor Schedule Database (Save Schedule; receives Schedule Data).
4.0 System Administration	Receives Manage Doctors/Patients and Manage Doctor Schedule from Admin. Returns Management Confirmation to Admin. Also sends Update User Records to Patient. Receives View Appointment Data (shown flowing at the bottom of the diagram).

Data Stores:

Data Store	Interactions as Shown in Diagram
D1 User Database	Interacts with Process 1.0 (User Authentication). Receives Store/Verify User requests; returns User Data to the process.
D2 Appointment Database	Interacts with Process 2.0 (Appointment Management). Receives Save/Update Appointment; returns Appointment Details.
D3 Doctor Schedule Database	Interacts with Process 3.0 (Doctor Schedule Management). Receives Save Schedule; returns Schedule Data.

Data Flows Summary:

Data Flow Label	Source → Destination
Registration / Login Data	Patient → Process 1.0
Authentication Result	Process 1.0 → Patient
Store / Verify User	Process 1.0 → D1 User Database
User Data	D1 User Database → Process 1.0
Book / Cancel Appointment	Patient → Process 2.0
Appointment Confirmation	Process 2.0 → Patient
Available Time Slots	Process 2.0 → Patient
Save / Update Appointment	Process 2.0 → D2 Appointment Database
Appointment Details	D2 Appointment Database → Process 2.0
Update Availability	Doctor → Process 3.0
Schedule Confirmation	Process 3.0 → Doctor
Save Schedule	Process 3.0 → D3 Doctor Schedule Database
Schedule Data	D3 Doctor Schedule Database → Process 3.0
Manage Doctors / Patients	Admin → Process 4.0
Management Confirmation	Process 4.0 → Admin
Manage Doctor Schedule	Admin → Process 4.0
Update User Records	Process 4.0 → Patient
View Appointment Data	Admin → Process 4.0 (shown at bottom of DFD)

This concludes the Software Requirements Specification for the Hospital Appointment Management System (Version 1.0). All sections conform to IEEE Std 830-1998.